

অপারেটিং সিস্টেমের ভূমিকা

(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবেরো

(Prof. Chester Rebeiro)

কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ

(Department of Computer Science and Engineering),

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি, মাদ্রাজ

(Indian Institute of Technology, Madras)

সপ্তাহ - 05

বক্তৃতা - 20

মাল্টিপ্রসেসর শিডিউলিং

(Multiprocessor Scheduling)

হ্যালো, এখন পর্যন্ত আমরা দেখেছি কিভাবে একটি একক CPU-র জন্য শিডিউলিং অ্যালগরিদম (scheduling algorithm) নির্ধারণ করা হয়। সেখানে শিডিউলিং অ্যালগরিদমটি একটি প্রসেসকে একটি CPU তে এক্সিকিউট (execute) করার জন্য নির্বাচন করেছিল। এই ভিডিওটিতে আমরা মাল্টিপ্রসেসর শিডিউলিং অ্যালগরিদম (multiprocessor scheduling algorithm) নিয়ে আলোচনা করব।

(স্লাইড সময় : 00:35 দেখুন)

মূলত, আমরা দেখব যে সিস্টেমে একাধিক প্রসেসর (processor) থাকলে কীভাবে একটি শিডিউলিং অ্যালগরিদম এই বিভিন্ন CPU-র মধ্যে প্রসেসগুলিকে এক্সিকিউট (execute) করার জন্য নির্বাচন করতে

পারে।

(স্লাইড সময় : 00:45 দেখুন)

যদি scheduler টিকে চালনা করার জন্য একটি নির্দিষ্ট (dedicated) CPU থাকে তাহলে একাধিক প্রসেসর scheduling কাজটি অনেক সহজ হয়। এই scheduler টি অপর প্রত্যেকটি CPU এর জন্য সিদ্ধান্ত নিতে পারে,

বিশেষ করে কোন প্রসেস কোন CPU তে এক্সিকিউট করা উচিত, তার সিদ্ধান্ত নেয়। এই প্রকল্পটি বাস্তবায়িত করা খুবই সহজ। এই পদ্ধতিতে, যে প্রসেসগুলি এক্সিকিউট করার জন্য প্রস্তুত (ready) থাকে সেগুলিকে প্রসেসরের একটি লোকাল কিউতে (local queue) রাখা হয়। এবার একটি নিয়ম অনুযায়ী কোন প্রসেস কোন CPU তে এক্সিকিউট করবে তা নির্দিষ্ট করা হয়। এই পদ্ধতিতে নিঃসন্দেহে কার্যক্ষমতা হ্রাস (performance degradation) হওয়ার সম্ভাবনা রয়েছে। অপর সবকয়টি CPU পরবর্তী কোন প্রসেসটি এক্সিকিউট করতে হবে তা জানার জন্য scheduler CPU টির উপর নির্ভরশীল। এই সিদ্ধান্ত খুব ঘন ঘন (মনে করুন প্রতি ১০ মিলিসেকেন্ডে) নেওয়া হয়। ফলে মূল কার্যহীন সময় (overhead) বৃদ্ধি পায়।

(স্লাইড সময়: 01:37 দেখুন)

আরেকটি মাল্টিপ্রসেসর শিডিউলিং পদ্ধতি হলো প্রতিসম শিডিউলিং পদ্ধতি বা স্কিম (symmetrical scheduling scheme)। এই পদ্ধতিতে একটি নির্দিষ্ট CPU এর পরিবর্তে প্রতিটি CPU স্বাধীনভাবে নিজস্ব scheduler চালনা করে। এর ফলে প্রতিটি CPU নির্দিষ্ট সময় (time slice) অন্তে পরবর্তী কোন প্রসেসটি এক্সিকিউট করবে সেই সিদ্ধান্ত নেয়। প্রতিসম শিডিউলিং-এর দুইটি বিকল্প হতে পারে – একটিতে প্রসেসগুলিকে একটি গ্লোবাল কিউতে (global queue) রাখা হয়, অপরটিতে প্রসেসগুলিকে প্রতিটি CPU এর স্বতন্ত্র সারিতে – পার CPU কিউতে (per CPU queue) রাখা হয়। এবার আমরা এই বিকল্পগুলি দেখব।

(স্লাইড সময়: 02:16 দেখুন)

গ্লোবাল কিউ (global queue) সম্পন্ন প্রতিসম শিডিউলিং (symmetric scheduling) পদ্ধতিতে, প্রস্তুত (ready) প্রসেসগুলিকে একটিমাত্র সারিতে রাখা হয়, যা প্রতিটি CPU ব্যবহার করে। প্রতিটি CPU এর scheduler এই গ্লোবাল সারি (global queue) দেখে নির্ধারণ করে যে কোন প্রসেসটিকে পরবর্তী এক্সিকিউশনের (running) জন্য নির্দিষ্ট করা হবে। যেমন, CPU 0 কে গ্লোবাল কিউ দেখে নির্ণয় করতে হবে যে কোন নির্দিষ্ট প্রসেসটি পরবর্তীতে এক্সিকিউশনে যাবে। সেই প্রসেসটি এবার প্রস্তুত অবস্থা (ready state) থেকে রানিং অবস্থায় (running state) যাবে। এই স্কিমের সুবিধা হল এই যে এটির ফলে CPU গুলির ব্যবহার্যতা (utilization) বৃদ্ধি পায় ও সমস্ত প্রসেসের মধ্যে নিরপেক্ষতা (fairness) বজায় থাকে। এই স্কিমটির একটি অসুবিধা হল এই যে, সিস্টেমের বিভিন্ন প্রসেসরগুলি একটিমাত্র সাধারণ কিউ ব্যবহার করে। এই কারণে এমন একটি অবস্থা হতে পারে যে কোন এক মুহূর্তে দুইটি প্রসেসর একই সাথে গ্লোবাল কিউটি পরীক্ষা (access) করে একই প্রসেসকে এক্সিকিউট করার জন্য নির্বাচন করেছে। ফলে একটিই প্রসেস একই সময়ে দুইটি পৃথক CPU তে এক্সিকিউট করবে। এই ধরনের সমস্যাগুলি রোধ করার জন্য গ্লোবাল কিউতে CPU গুলির পরীক্ষা (access) ক্রমানুযায়ী (serially) হওয়া প্রয়োজন। যেমন, CPU 0 যখন কিউটি পরীক্ষা (access) করছে, সেই মুহূর্তে অন্য কোন CPU কিউটি পরীক্ষা (access) করতে পারবে না। সুতরাং এটি নিশ্চিত হবে যে কোন নির্দিষ্ট সময়ে কেবল মাত্র একটি CPU গ্লোবাল

কিউটিকে পরীক্ষা (access) করে প্রসেস নির্বাচন করছে। সাধারণ কিউটিকে এইভাবে ক্রমানুসারে (serially) পরীক্ষা করার যে পদ্ধতিতে তাকে লকিং (locking) বলা হয়। এই লকিং প্রক্রিয়া নিঃসন্দেহে কার্যকারী, কিন্তু এর একটি প্রধান সমস্যা হল এই যে এই প্রক্রিয়ায় কর্মক্ষমতা প্রসারণের (scalability) সুযোগ কম, অর্থাৎ প্রসেসর সংখ্যা যত বৃদ্ধি পাবে লকিং এর ফলে তত সময় নষ্ট হবে। ধরা যাক 4 টি প্রসেসরের বদলে 64 টি প্রসেসর রয়েছে এবং প্রত্যেকে গ্লোবাল কিউকে (global queue) ক্রমানুযায়ী পরীক্ষা (access) করে। এখন যদি CPU 0 গ্লোবাল কিউকে (global queue) পরীক্ষা করে তাহলে সেই সময়ে অন্য CPU গুলি অপেক্ষা করে থাকবে। এর ফলে শিডিউলিং এর জন্য অনেক সময় ব্যয় হবে। আরেকটি অসুবিধা এই যে প্রসেসর আসক্তি – প্রসেসর অ্যাফিনিটি (processor affinity) থাকলে তা সহজে অর্জিত হয় না। এখন, প্রসেসর অ্যাফিনিটি কি? সিস্টেম যে পদ্ধতিতে কোন ব্যবহারকারীকে (user) তার প্রসেসটি কোন প্রসেসরে এক্সিকিউট করবে তা বেছে নিতে দেয় তাকে প্রসেসর অ্যাফিনিটি (processor affinity) বলে। যেমন কোন ব্যবহারকারী চাইতে পারেন যে তাঁর প্রসেসটি CPU 0 ছাড়া অন্য কোন CPU তে এক্সিকিউট করবে না। কিন্তু এই পদ্ধতিতে প্রসেসর অ্যাফিনিটির অনুমোদন দেওয়া কঠিন। এই স্কিমটি Linux 2.4 Kernel এবং xv6 অপারেটিং সিস্টেমে ব্যবহার হয়।

(স্লাইড সময় : 05:12 দেখুন)

প্রতিসম শিডিউলিংয়ের (symmetric scheduling) দ্বিতীয় প্রকারটিতে প্রতিটি CPU- এর নিজস্ব আলাদা প্রস্তুত প্রসেসর সারি (ready queue) থাকে। প্রতিটি CPU শুধুমাত্র তার নিজস্ব সারিকে পরীক্ষা (access) করেই নির্ধারণ করতে পারে কোন প্রসেসটি এক্সিকিউট করবে। এই কৌশলটিতে, প্রথমেই প্রস্তুত প্রসেসগুলিকে কতগুলি স্থির বিভাগে বিভক্ত করা হয় এবং ব্যবহারকারী বা অপারেটিং সিস্টেম স্থির করে নেয় যে কোন প্রসেসটি কোন CPU তে এক্সিকিউট করবে। সেই অনুযায়ী প্রসেসটিকে সংশ্লিষ্ট CPU কিউতে (CPU queue) রাখা হয়। সুতরাং দেখা যাচ্ছে যে এটি প্রয়োগ করা খুবই সহজ, কোন লকিং হয় না এবং কর্মক্ষমতা প্রসারণের (scalability) সুযোগ বজায় থাকে কারণ কোন CPU-এর অপর কোন CPU-এর সারি (queue) সম্বন্ধে চিন্তা করতে হয় না। কর্মক্ষমতা হ্রাস পায় না। সুবিধাটা হল এটা বাস্তবায়ন করা সহজ। লকিং পদ্ধতি না থাকায় বা CPU এর কিউগুলিকে ক্রমানুযায়ী পরীক্ষা করার না থাকায় scalability বজায় থাকে। অপর সুবিধাটা হল স্থানীয়তা। CPU গুলি স্থানীয় ভাবে, অপর CPU সারির ব্যাপারে চিন্তা না করেই প্রসেস নির্বাচন করতে পারে। যেমন, CPU 0-কে কেবলমাত্র নিজের সারির দিকে তাকালেই চলে সিস্টেমের অন্য কোনও কিউয়ের ব্যাপারে না জানলেও চলবে। এই কারণে প্রসেস নির্বাচন দ্রুত হয় এবং কর্মক্ষমতা হ্রাসের (performance degradation) সম্ভবনা কমে। এই স্কিমটির অসুবিধা হল এই যে বিভিন্ন CPU-এর দায়িত্ব (load) বিভিন্ন হয়ে সিস্টেম ভারসাম্যহীন হয়ে পড়তে পারে। অর্থাৎ, এমন হতে পারে যে কোন কোন CPU (যেমন CPU 2) এর প্রস্তুত সারিতে প্রচুর প্রসেস রয়েছে

আবার অন্যান্য CPU গুলির (যেমন CPU 3) কিউগুলি ফাঁকা রয়েছে। এর ফলে CPU 2 এর লোড CPU 3 এর তুলনায় অনেক বেশি হয়ে পড়ে।

(স্লাইড সময়: 07:29 দেখুন)

প্রতিসম শিডিউলিং (symmetric scheduling) তৃতীয় পদ্ধতিটি হচ্ছে মিশ্র (hybrid) পদ্ধতি। 2.6 বা তদুর্ধ্ব Linux Kernel এ পদ্ধতিটি ব্যবহার করা হয়েছে। মূলত এই পদ্ধতিতে স্থানীয় সারির (local queue) পাশাপাশি সাধারণ (global queue) সারি উভয়ই ব্যবহার করা হয়। স্থানীয় সারিগুলি প্রতিটি CPU এর সাথে যুক্ত থাকে। প্রতিটি CPU- এর নিজস্ব স্থানীয় সারি (local queue) থাকে এবং সাধারণ সারিটিকে সমস্ত CPU গুলি ব্যবহার করে। সাধারণ সারিটি লোডের ভারসাম্য বজায় রাখে এবং এটি নিশ্চিত করে যে CPU গুলিতে যাতে মোটামুটি সমসংখ্যক প্রসেস এক্সিকিউট করে। স্থানীয় সারিগুলি কর্মক্ষমতা যাতে হ্রাস না পায় সেটি নিশ্চিত করে।

(স্লাইড সময় : 08:23 দেখুন)

লোড ভারসাম্য (load balancing) অর্জনের জন্য সাধারণ সারির পাশাপাশি দুটি আরও কৌশল রয়েছে - একটি পুশ মাইগ্রেশন (Push Migration) এবং অন্যটি পুল মাইগ্রেশন (Pull Migration)। পুশ মাইগ্রেশন প্রক্রিয়াতে, একটি বিশেষ টাস্ক (task) কিছু সময় অন্তরে প্রসেসরগুলির লোড পর্যবেক্ষণ করে এবং লোডের ভারসাম্যহীনতা দেখা গেলে প্রসেসরগুলিকে প্রসেসরগুলির মধ্যে পুনঃবিতরণ (redistribute) করে। পুল মাইগ্রেশন প্রক্রিয়াতে, একটি নিষ্ক্রিয় প্রসেসর (idle processor) অপর কোন ব্যস্ত প্রসেসরের থেকে প্রসেস টেনে নিয়ে সেটিকে এক্সিকিউট করে। এইভাবে বেশী ব্যস্ত প্রসেসর থেকে কম ব্যস্ত প্রসেসরে প্রসেস স্থানান্তরের ফলে লোডের ভারসাম্য বজায় থাকে। এই প্রসেস স্থানান্তর প্রয়োজন বুঝে করা উচিত। একটি CPU থেকে অপর CPU তে স্থানান্তরের প্রক্রিয়াটি বেশ ব্যয়বহুল। যখনই একটি প্রসেস এক CPU থেকে অন্য CPU তে স্থানান্তরিত হয়, তখনই নতুন CPU- র সাথে সম্পর্কিত সমস্ত মেমরি পুনঃপ্রতিষ্ঠা (memory repopulation) করতে হয়। অর্থাৎ, নতুন CPU- র সাথে যুক্ত ক্যাশ মেমরিতে (cache memory) স্থানান্তরিত প্রসেসটির নির্দেশাবলী (instructions) এবং তথ্য (data) লোড করতে হয়। অনুরূপভাবে, টিএলবি (TLB) খালি করতে হয়, ইত্যাদি। এর ফলে অনেক কার্যহীন সময় ব্যয় হয়। এই কারণে প্রসেস স্থানান্তর শুধুমাত্র প্রয়োজনেই করা উচিত। এর সাথে আমরা এই ভিডিওটি শেষ করব। আমরা এই ভিডিওটিতে আমরা বিভিন্ন CPU এর মাল্টিপ্রসেসর শিডিউলিং ও লোড ভারসাম্যতা নিয়ে আলোচনা করেছি। ধন্যবাদ।